MISE EN PLACE D'UNE APPLICATION SERVERLESS DE TRAITEMENT D'IMAGES AVEC AWS LAMBDA ET S3

La formation de 45 minutes de Amazon Skills Builder se base sur l'apprentissage de ces points :

- Créer une fonction AWS Lambda
- Configurer un compartiment Amazon S3 en tant que source d'événement Lambda
- Déclencher une fonction Lambda en chargeant un objet dans Amazon S3
- Contrôler les fonctions AWS Lambda S3 via Amazon CloudWatch Logs.

C'est quoi AWS Lambda ?

AWS Lambda est un service de calcul serverless qui permet **d'exécuter du code automatiquement en réponse à des événements**, sans avoir à gérer de serveur.

Concrètement :

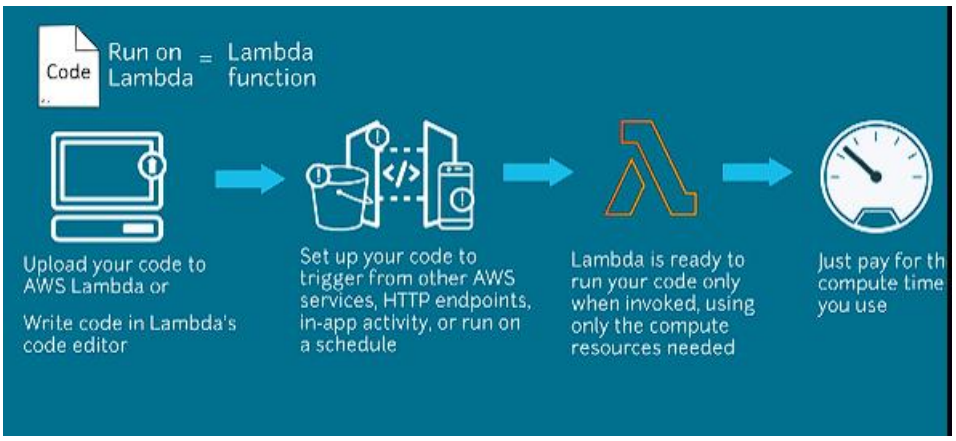
- On écrit du code
- AWS s'occupe de l'exécution
- Le code s'exécute uniquement lorsqu'un événement se produit

Exemple d'événements :

- Upload d'un fichier
- Requête API
- Action utilisateur

Comment ça marche ?

AWS Lambda fonctionne sur 4 étapes :



1. Nous **devons uploader notre code sur AWS Lambda** ou bien le taper dans l'éditeur de texte AWS Lambda. A ce stade le code est stocké mais ne s'exécutera pas.
2. **Configuration des triggers (déclencheur) :** chaque fonction du code s'exécute en fonction d'un évènement précis. Que ce soit via une requête API, un évènement dans la base de données ou une planification.
3. **Activation :** Dès que le déclencheur (étape 2) est activé, AWS alloue instantanément les ressources nécessaires (CPU, RAM). Lambda va exécuter notre code dans un environnement sécurisé et isolé.

Par exemple : si 100 personnes activent un évènement de notre code en même temps, 100 instances de notre fonction seront lancées simultanément sans qu'on ait à gérer les serveurs.

4. **Pay for the compute time you use :** on est facturé uniquement sur temps de compilation (millisecondes) qu'on utilise ainsi que les ressources. Donc si personne n'utilise l'application on sera facturé 0 euros.

Les avantages de AWS Lambda ?

Nous n'avons pas de serveurs à gérer.

Nous devons juste écrire notre code et Lambda s'occupe de le gérer.

On paie seulement les ressources et le temps utilisé.

TABLE DES MATIERES

introduction a amazon lambda ..... **Erreur ! Signet non défini.**
VOCABULAIRE 2
exploration de aws lambda 4
créer les compartiments amazon s3 5
créer une fonction lambda 8
tester la fonction 13
conclusion 15

LES NOTIONS TRAVAILLEES

AWS Lambda (serverless) → exécuter du code sans gérer de serveur

Amazon S3 (stockage cloud) → stockage d'objets (images, fichiers, etc.)

Événements (ObjectCreated) → déclenchement automatique lors d'un upload

Triggers (déclencheurs)→ lien entre S3 et Lambda

Fonction Lambda (Python)→ traitement d'image (redimensionnement)

IAM Role (permissions)→ autoriser Lambda à accéder à S3

Architecture serverless→ automatisation sans infrastructure à gérer

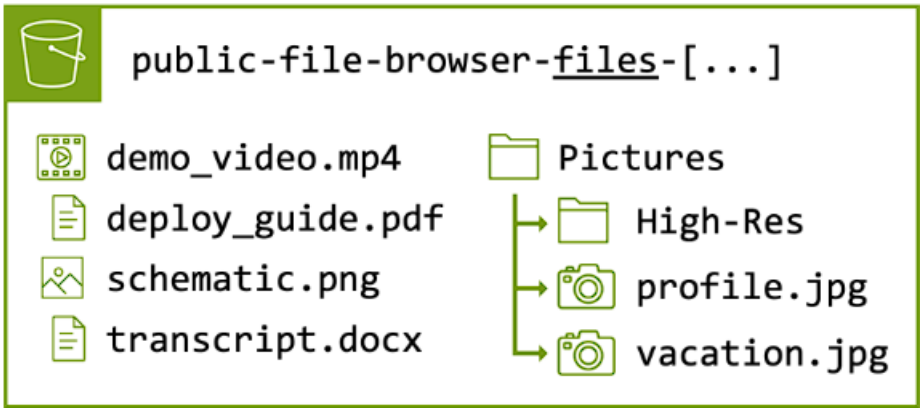
Tests avec événements JSON→ simulation d'un déclenchement

VOCABULAIRE

1. **Bucket (compartiment)**

C'est un **espace de stockage dans AWS** où on pourra y stocker des objets (images, vidéos, fichiers Excel, documents, métadonnées, etc.). Il agit comme une source d'**événements**. Dès qu'un fichier est ajouté (ObjectCreated) ou supprimé, le bucket envoie un signal pour déclencher un traitement automatique.

Exemple :



Donc ça va créer un évènement ObjectCreated.

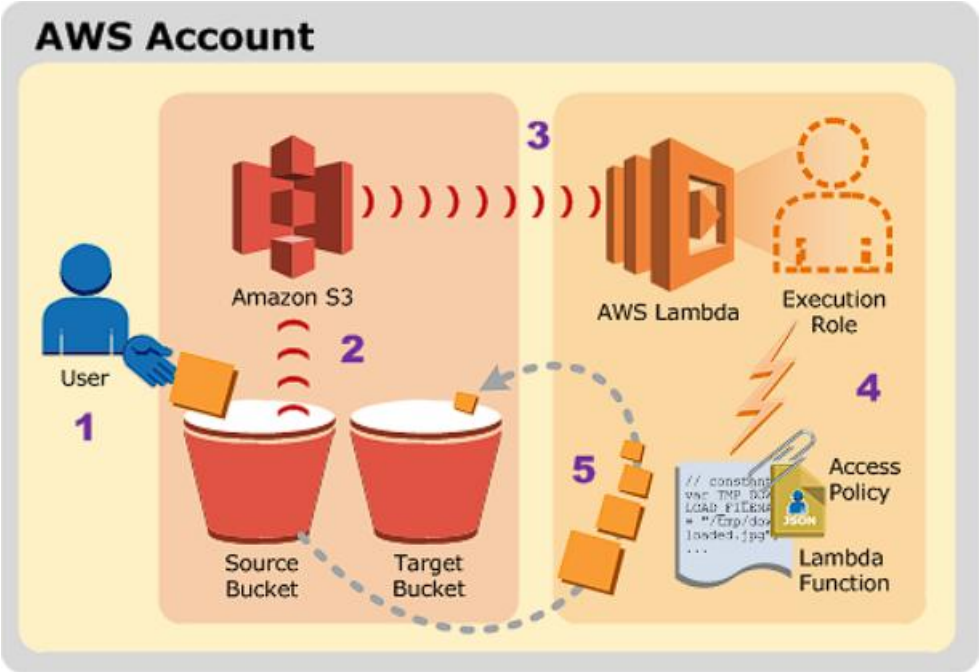
2. ***Amazon S3 (Simple Storage Services)***

C'est le service de **stockage d'objets** disponible et sécurisé d'AWS.

- **Rôle :** Il sert de "disque dur cloud" capable de stocker une quantité quasi-illimitée de données non structurées (images, logs, sauvegardes).
- **Particularité :** Contrairement à un serveur classique, on y accède via Internet (URL) et non par un système de fichiers local.

EXPLORATION DE AWS LAMBDA

Nous allons travailler sur un atelier AWS Lambda en créant une **application d'image miniature** sans serveur.



<u>Dans cette image :</u>

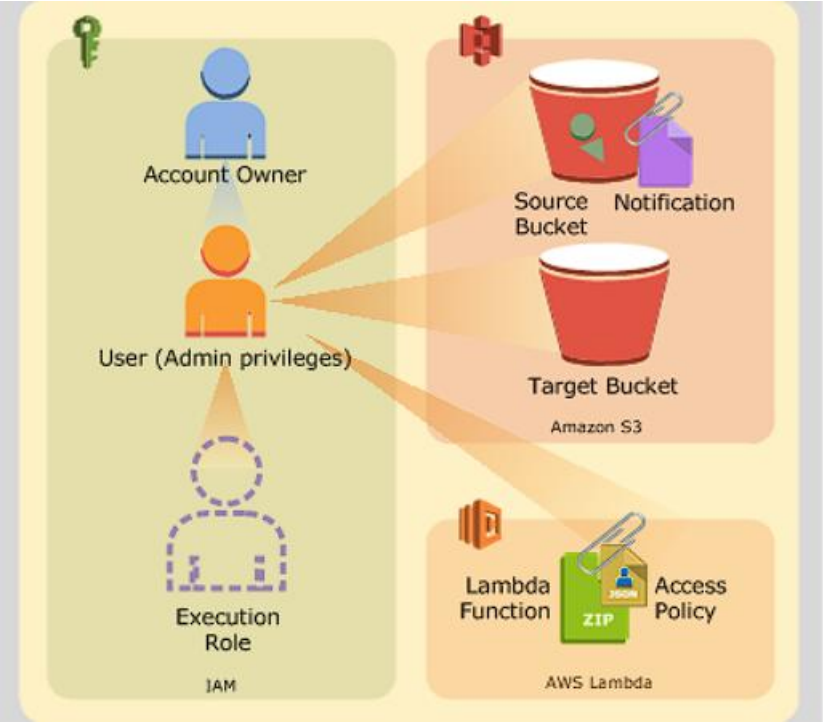
a. Un **utilisateur importe un objet dans un bucket** dans le compartiment source Amazon S3.
b. Amazon S3 va **détecter l'évènement de création d'objet** (ObjectCreated).
c. S3 envoie l'événement à Lambda : S3 va dire à Lambda qu'un fichier a été ajouté ou supprimé.
d. Lambda va exécuter le code de la fonction
e. La fonction Lambda va traiter le fichier :

Ton code utilise les infos reçues pour agir :

Exemple concret (comme dans ton image) :

- récupère le nom du fichier
- lit l'image
- crée une miniature (thumbnail)
- enregistre la miniature dans un autre bucket

À la fin de ce didacticiel, on aura les ressources suivantes :

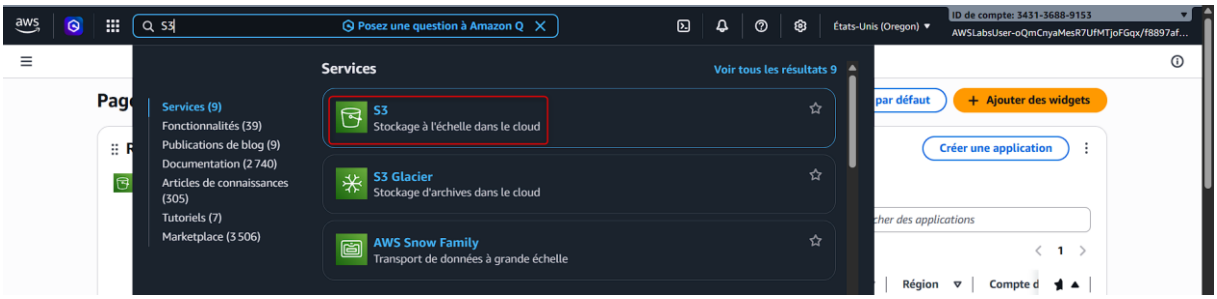


CREER LES COMPARTIMENTS AMAZON S3

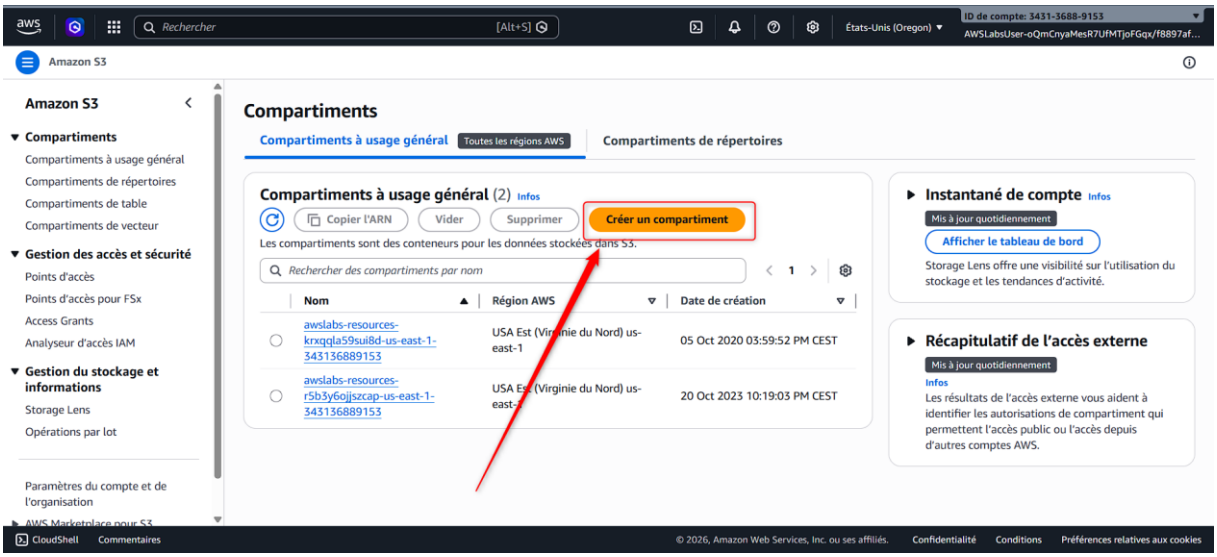
Pour le projet, nous allons **créer 2 compartiments** : 1 **compartiment d'entrée** pour insérer et stocker les photos, et **1 compartiment de sortie** pour sortir les photos redimensionnées.

En haut de la Console de gestion AWS, dans la barre de recherche unifiée, recherchez et **sélectionnez S3**

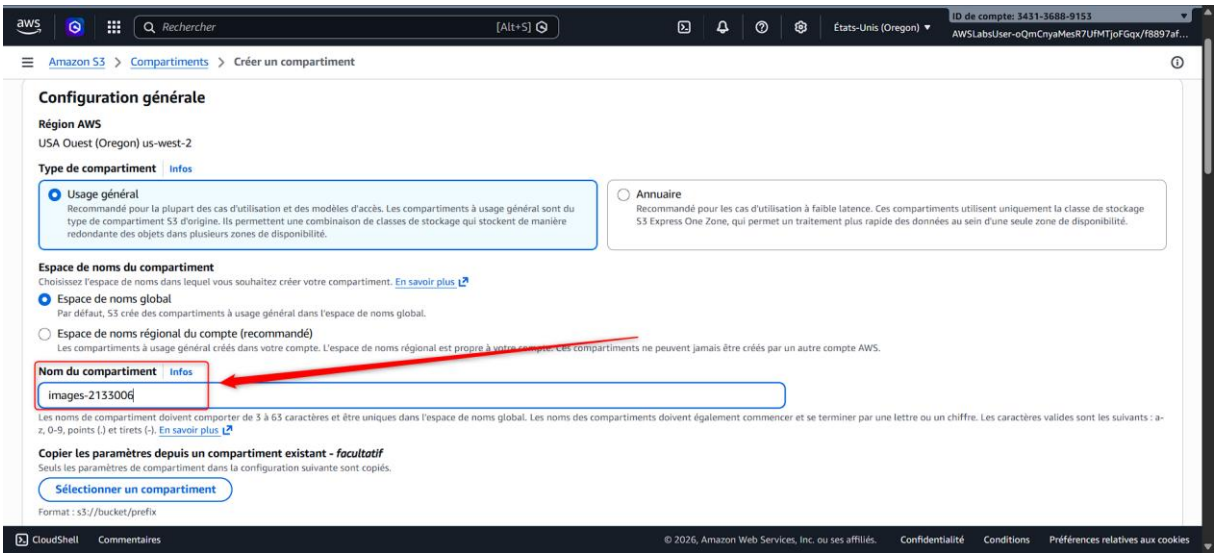
.



Cliquez sur **Créer un compartiment** :

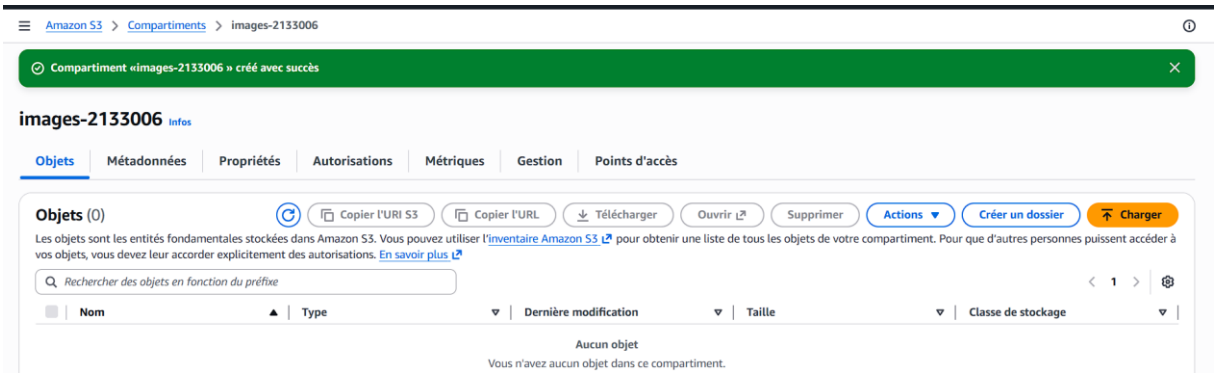


Donnez un nombre unique à votre bucket, dans notre cas images-2133006 .Laissez les paramètres par défaut puis descendez la page et cliquez sur **créer un compartiment** :

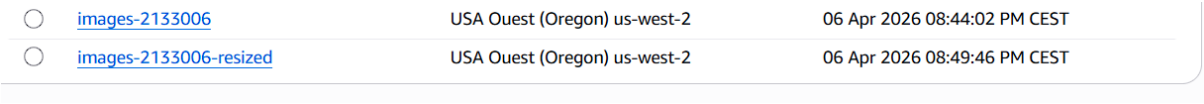


➔ Comme expliqué dans le vocabulaire, chaque nom de bucket doit être unique quel que soit la région.

Le compartiment a été créé avec succès :

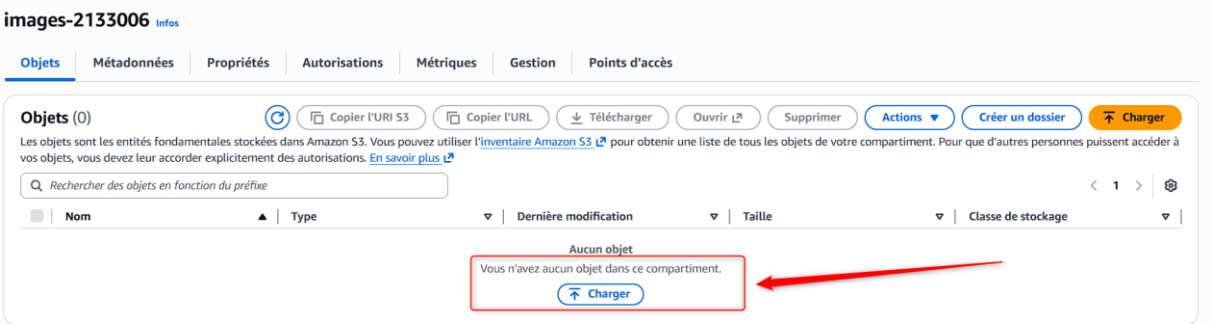


Créez un autre compartiment de sortie (images-2133006-resized) :



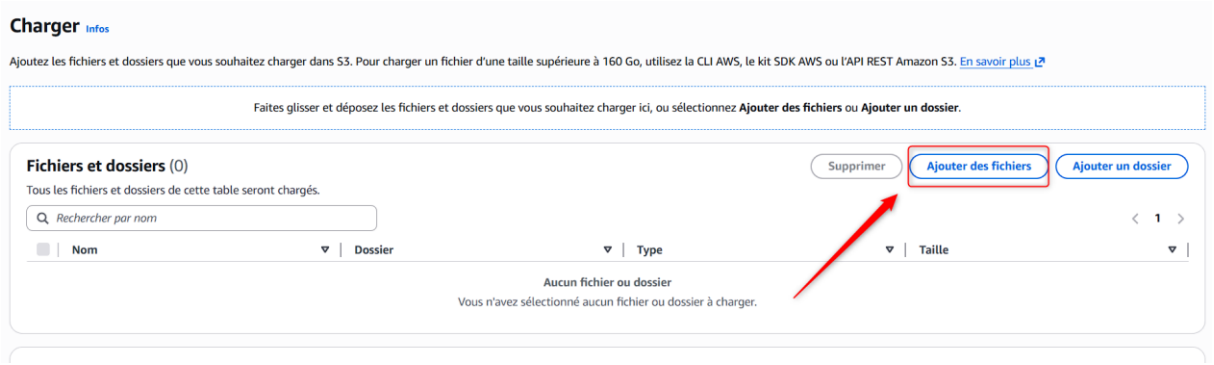
➔ Nous 2 compartiments ont été créés.

Sélectionnez une image de votre ordinateur que vous voulez importer, puis sélectionnez le compartiment d'entrée (images-2133006).

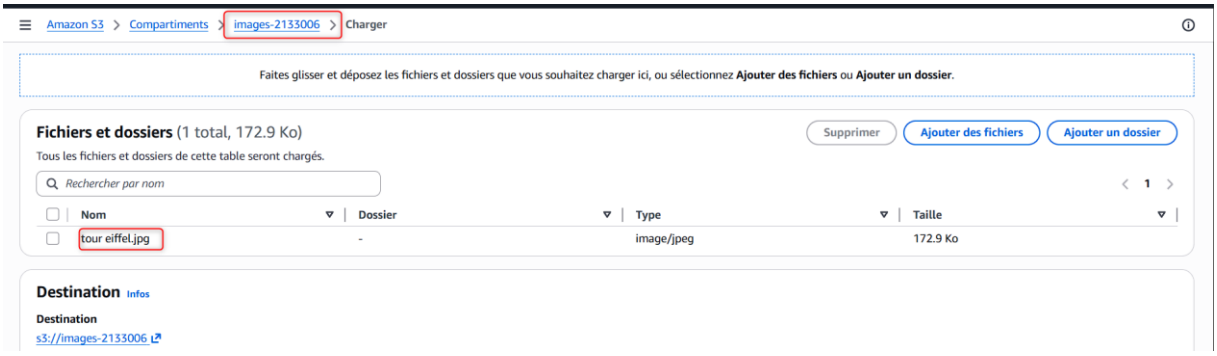


➔ Cliquez sur Charger

Cliquez sur **Ajouter des fichiers et ajouter votre image**. Dans notre cas, ce sera une image en 1024x500:



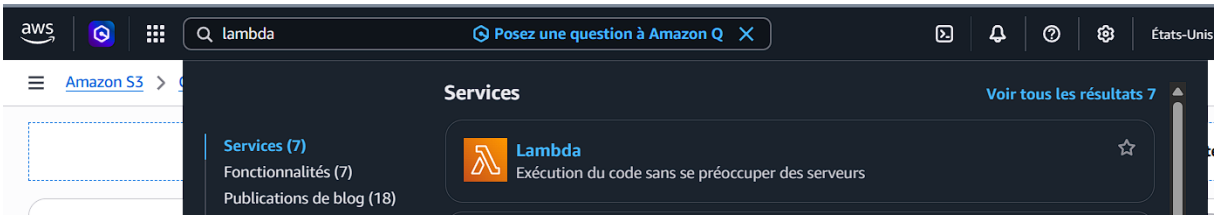
C'est bon ! L'image a été importé dans le compartiment :



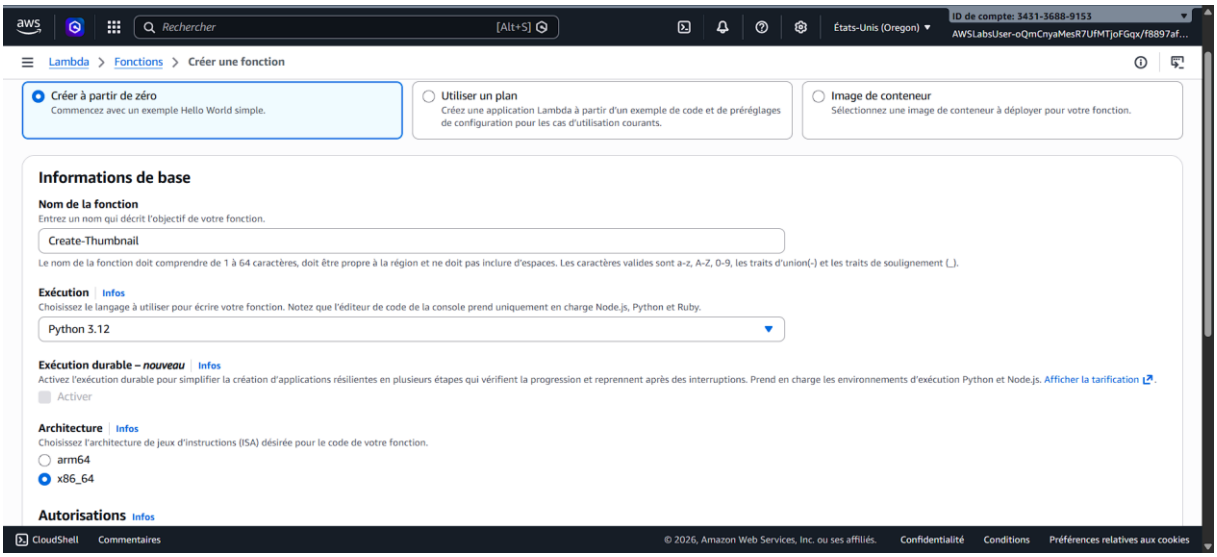
CREER UNE FONCTION LAMBDA

Nous allons maintenant passer à la deuxième étape qui consiste à **créer une fonction Lambda** qui va lire l'image qu'on a importé dans notre compartiment d'entrée (images-2133006/tour eiffeil.jpeg) puis qui va la redimensionner dans le répertoire (images-2133006-resized).

Dans la barre de recherche AWS, **recherchez Lambda** et **cliquez** :

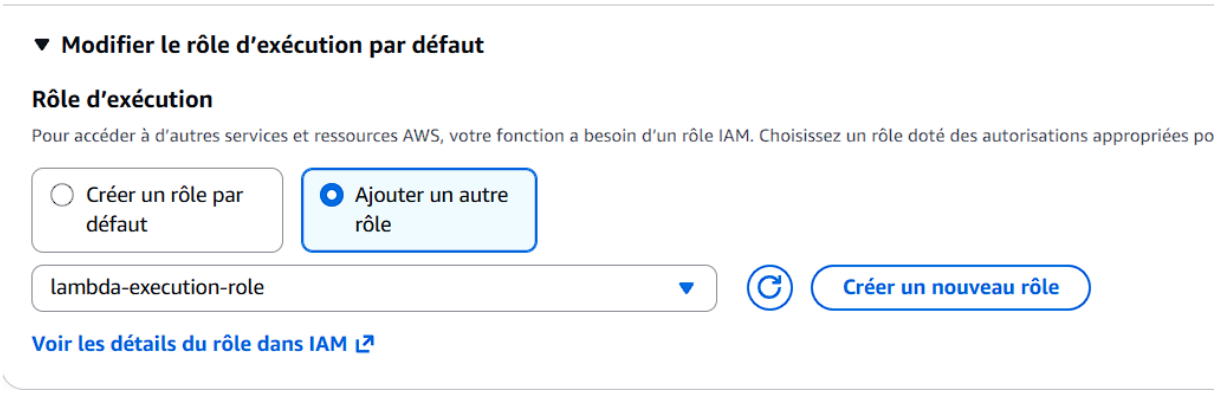


Une fois sur la page, **cliquez sur le bouton Créer une fonction** puis sélectionnez l'option **Créer à partir de zéro** :



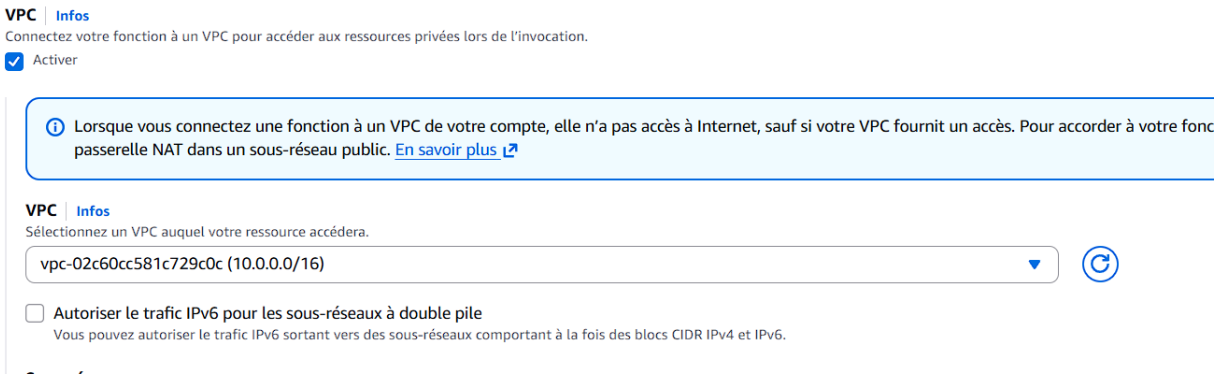
- ➔ Donnez-lui un nom : Create-Thumbnail
- ➔ Mettez le langage d'exécution en Python

Cliquez sur **Ajouter un autre rôle puis entrez lambda-execution-rôle** :

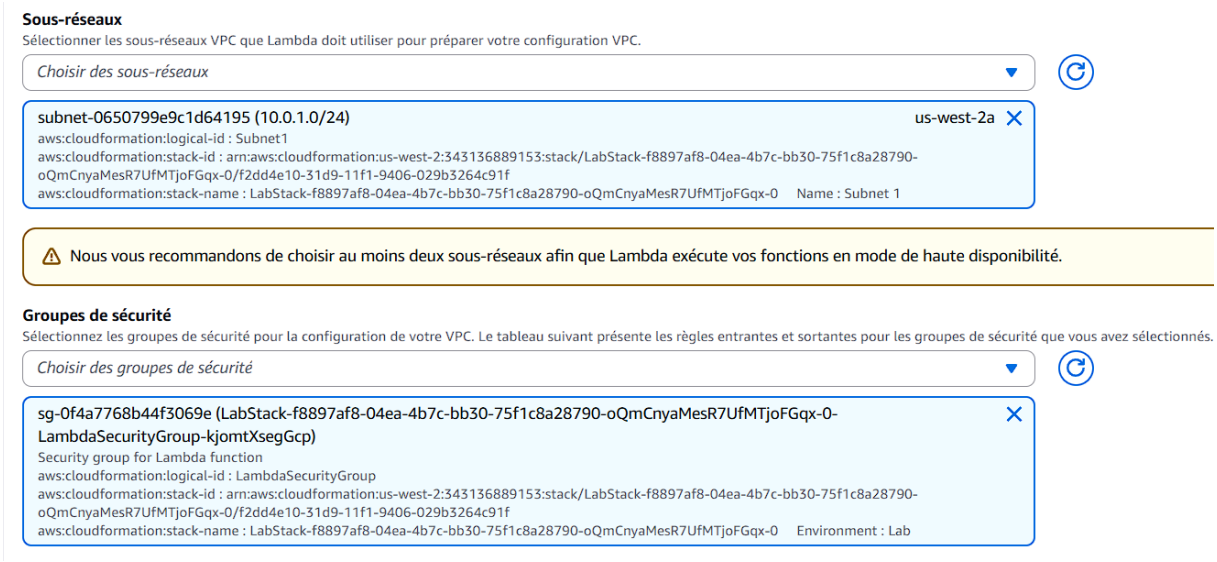


REMARQUE : ce rôle va permettre à Lambda d'entrer sans S3 et de lire et écrire sur nos images.

Déroulez la menue **information supplémentaire**, **cliquez sur VPC** et sélectionnez le CIDR (**10.0.0.0/16**) :



Sélectionnez les **sous-réseaux (10.0.1.0/24),** puis le **groupe de sécurité (LambdaSecurityGroup**) puis cliquer sur **Créer une fonction** :

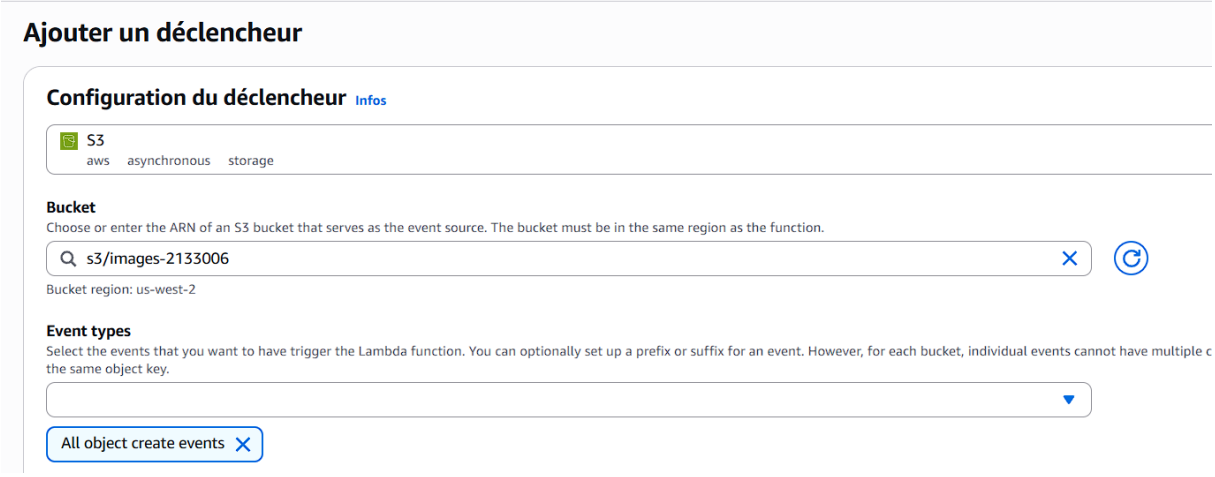


➔ Ces options vont nous permettre de lire les fichiers locaux.

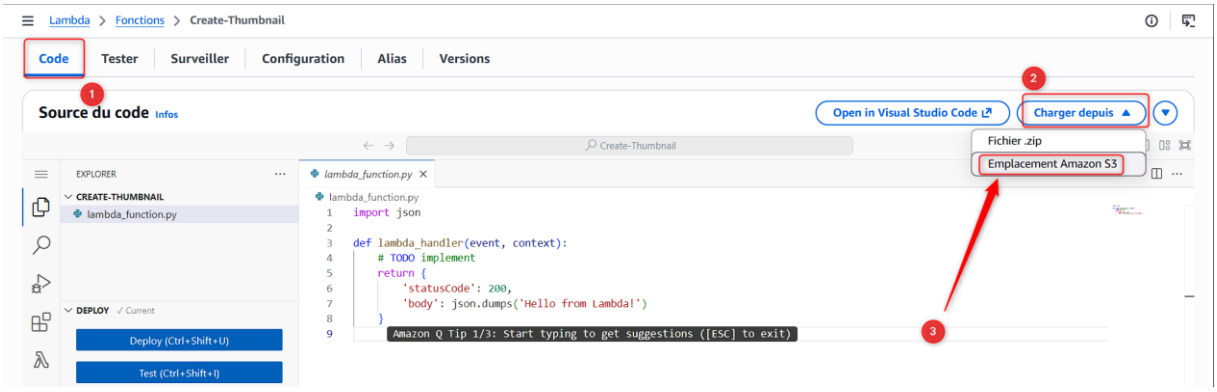
Attendez que la bannière du haut affiche :

Nous allons faire en sorte de créer la fonction qui agiras à chaque ajout d'objets dans notre compartiment d'entrée.

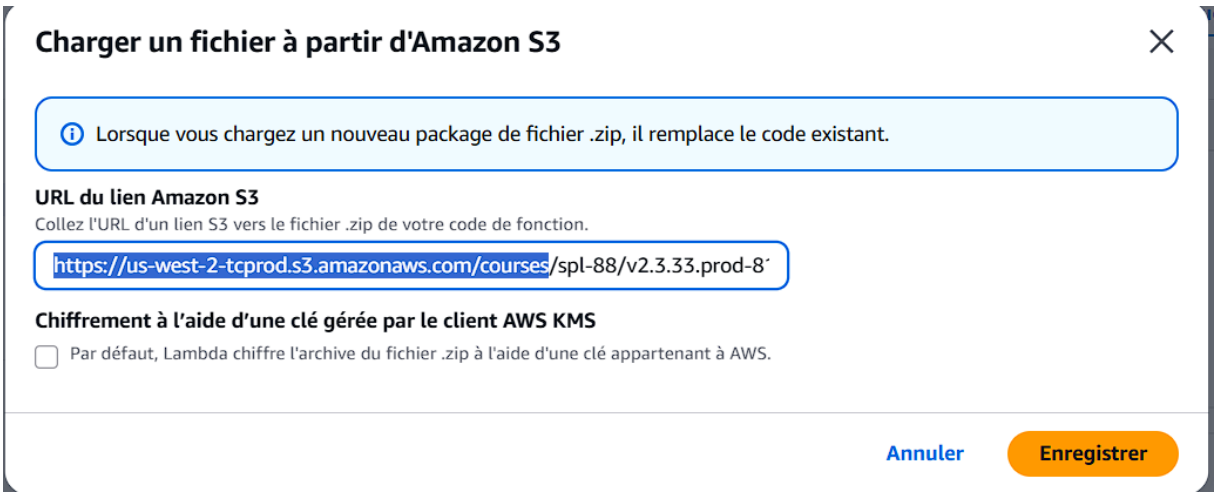
Cliquez sur **Ajouter un déclencheur**, tapez dans la barre **S3** puis **sélectionnez le compartiment d'entrée(images-2133006). Cochez la case « I acknowledge ... »** puis cliquez sur **Ajouter :**



Nous allons importer les fonctions prédéfinis. Allez dans **l'onglet Code**, cliquez sur **Charger depuis** puis dans **l'emplacement Amazon S3** :

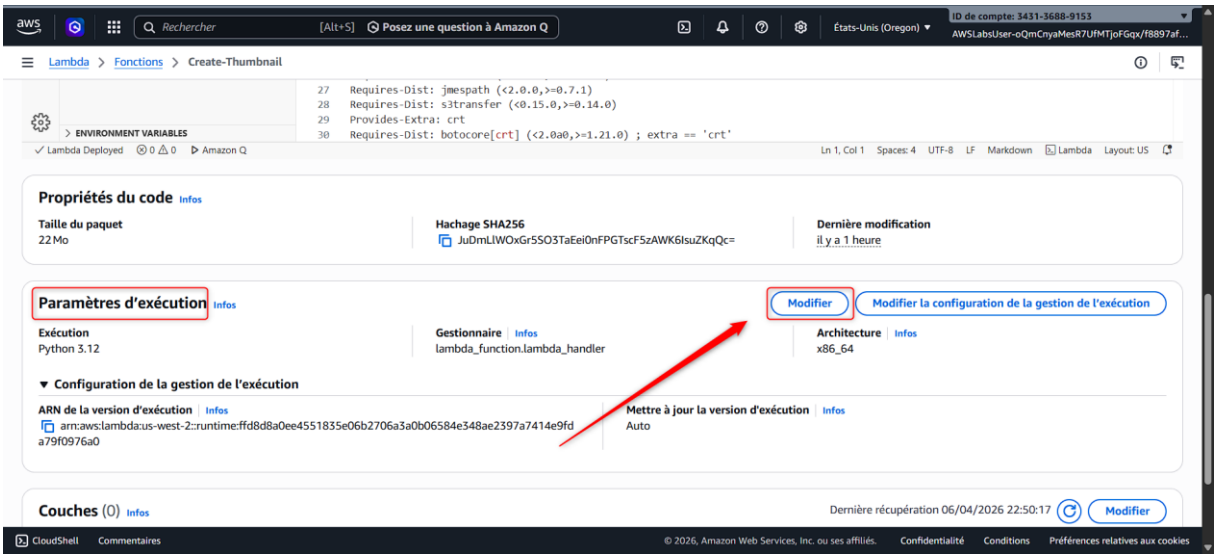


Dans l'URL tapez ça : https://us-west-2-tcprod.s3.amazonaws.com/courses/spl-88/v2.3.33.prod-812ae975/scripts/CreateThumbnail.zip

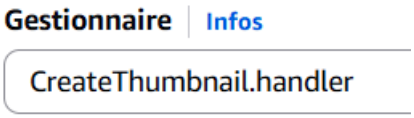


➔ Ce dossier contenant la fonction va : recevoir l'évènement de création d'objet, télécharger l'image depuis S3/images-resized, redimensionner l'image (librairie pillow), charger l'image dans (images-213306-resized).

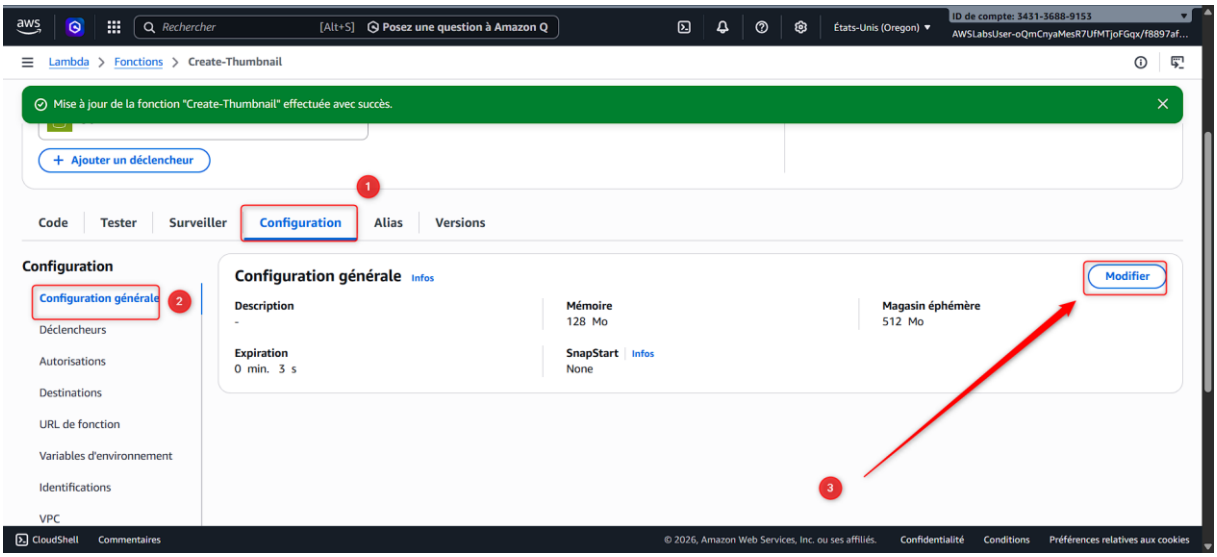
Dans la section **Runtime settings** (Paramètres d'exécution), sélectionnez **Modifier** .



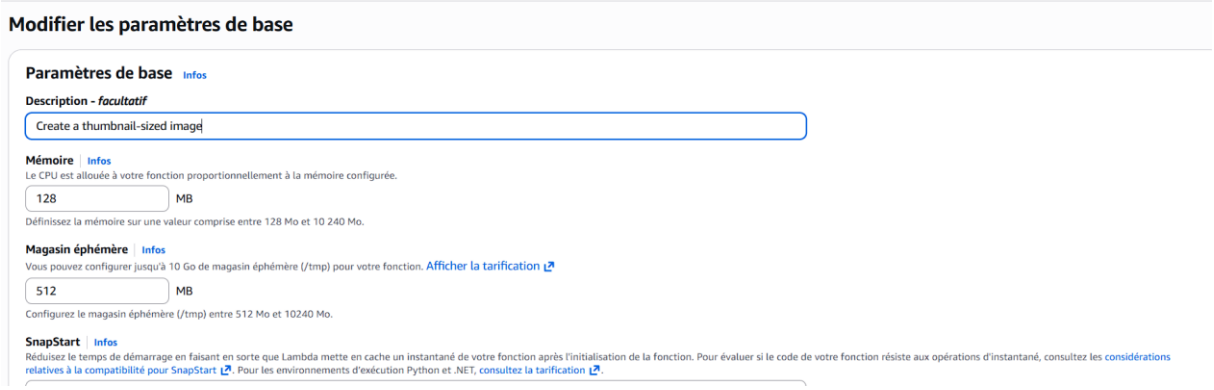
Dans gestionnaire mettez, puis cliquez sur Save :



Sur la gauche de l'écran, sélectionnez **General configuration** (Configuration générale), puis sélectionnez **Modifier** .



Entrez la description :

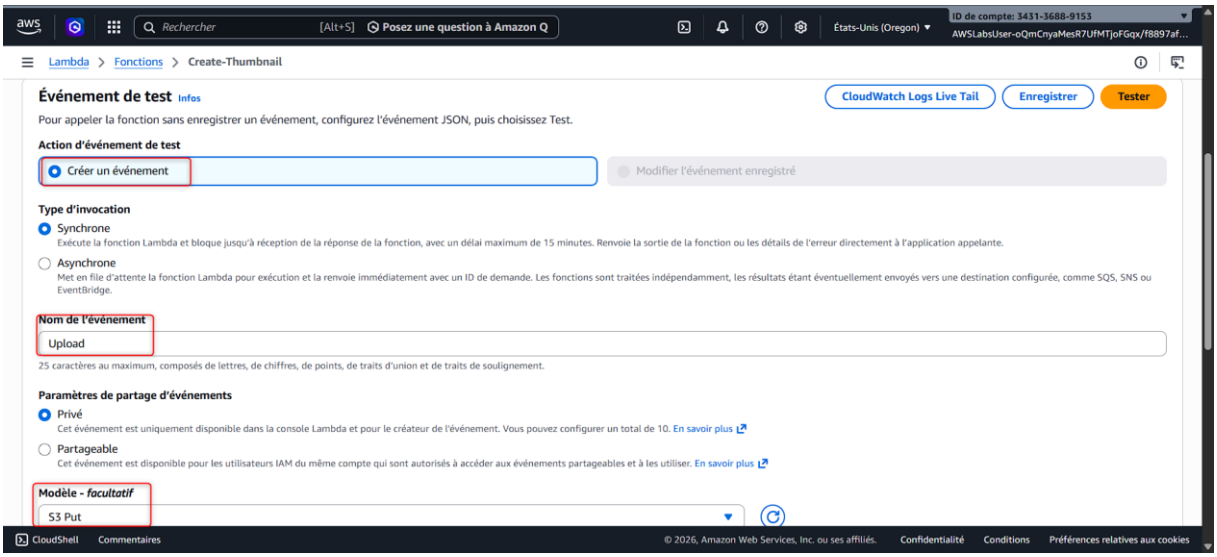


➔ La mémoire définit les ressources qui seront alloués à la fonction.

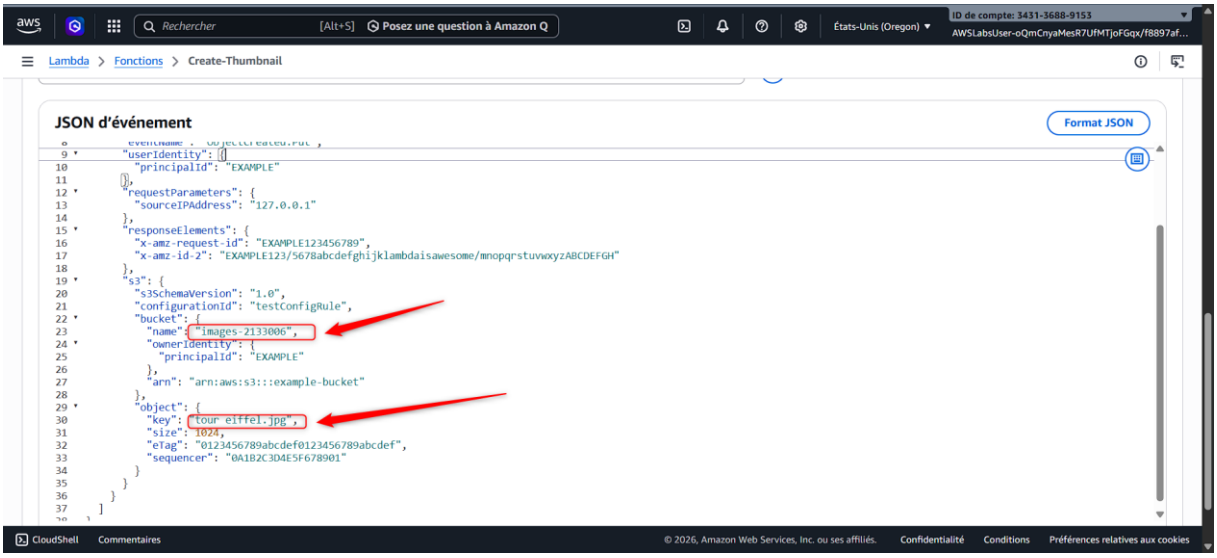
TESTER LA FONCTION

Cette tâche nous permettra de tester et simuler notre fonction.

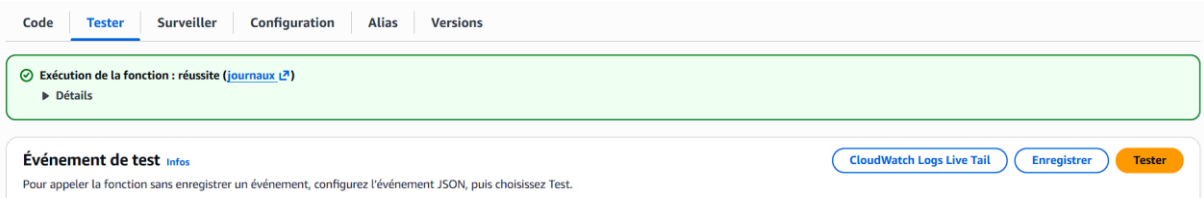
Allez dans **l'onglet Test** → évènement de test → **créer un évènement** → appelez l'évènement « **upload** » → Modèle facultatif : **S3 Put**



Descendez jusqu'à JSON d'événement → remplacez le contenu de la variable name par : images-2133006 (conteneur d'entrée) → remplacez le contenu de la variable key par le nom de votre image (tour eiffel .jpg)

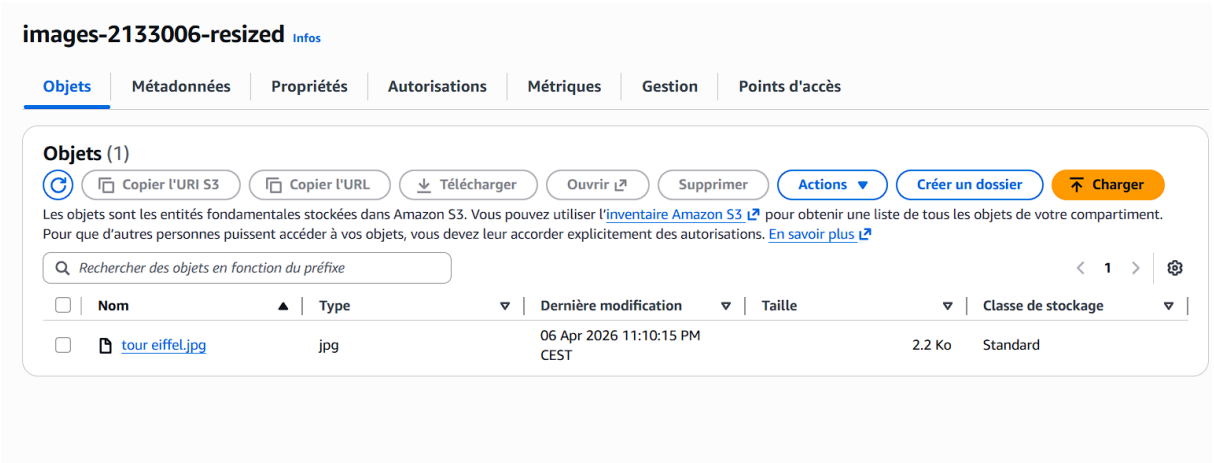


Cliquez sur Tester et l'exécution sera réussie :

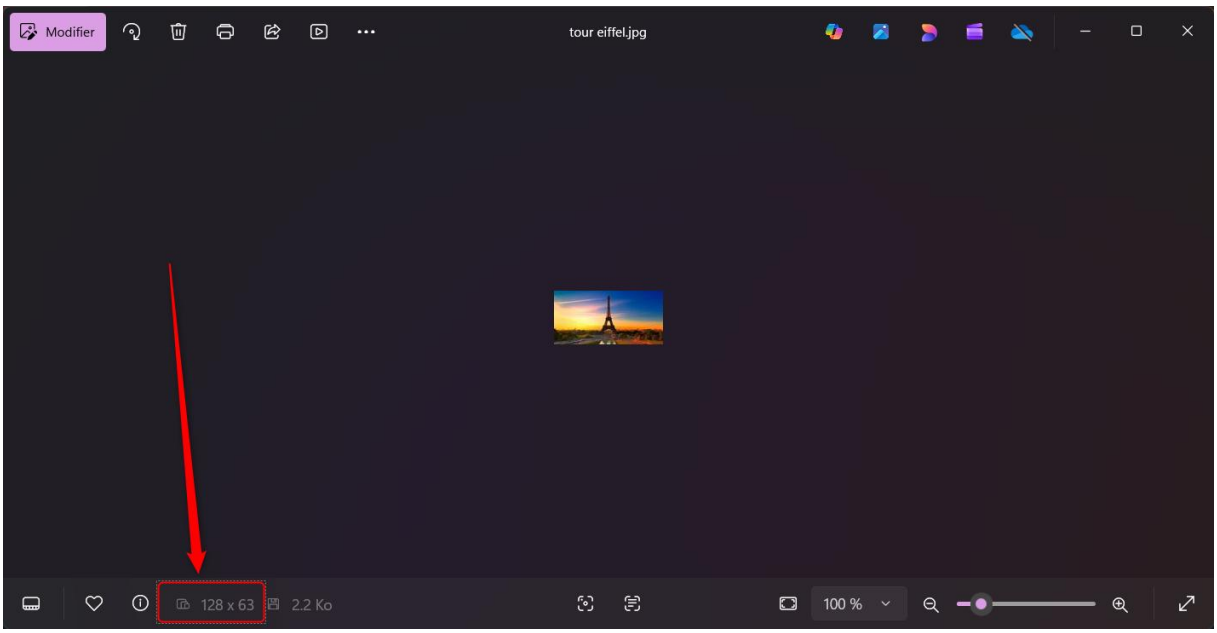


Tapez dans la barre de recherche S3 pour retourner vers votre centre de stockage.

Allez dans **votre bucket images-2133006-resized** et vous verrez que **votre image a été déplacé et redimensionné** ici :



Sélectionnez l'image et ouvrez-la. Vous verrez que l'image est miniature et redimensionnée (128x63) :



Voila !

Après exécution **l'image originale reste dans le bucket** source et une **image redimensionnée apparaît dans le bucket images-2133006-resized**.

CONCLUSION

Ce TP nous a permis de comprendre concrètement le fonctionnement d'une architecture **serverless** avec AWS.

Nous avons appris à relier plusieurs services cloud (S3, Lambda, CloudWatch) pour créer une application automatisée capable de traiter des données sans intervention humaine.

L'un des principaux avantages d'AWS Lambda est qu'il permet de ne pas gérer de serveur, automatiser des tâches efficacement, payer uniquement à l'usage.

Cette approche est aujourd'hui très utilisée dans le développement moderne, notamment pour les APIs , le traitement de données et les applications web évolutives

En conclusion, AWS Lambda est un outil puissant qui simplifie énormément le développement d'applications cloud modernes, tout en offrant flexibilité, performance et réduction des coûts.